Lab Exercise 18- Volume in Kubernetes Cluster

Volumes in Kubernetes provide a way to store and share data among containers in a Pod. This lab exercise will guide you through the creation and use of different types of volumes in Kubernetes.

Step 1: Set Up Kubernetes Cluster

Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

Step 2: Create a Pod with an emptyDir Volume

Create a file named **emptydir-pod.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-vol1
spec:
  containers:
    - image: nginx
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      emptyDir: {}
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % cat emptydir-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-vol1
spec:
  containers:
    - name: test-container
      image: nginx
      volumeMounts:
        - name: first-volume
          mountPath: /data
  volumes:
    - name: first-volume
      emptyDir: {}

shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

In this manifest:

The emptyDir volume is a temporary storage that is created when the Pod is assigned to a node and exists as long as the Pod is running.

Apply the manifest to create the Pod:

```
kubectl apply -f emptydir-pod.yaml
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl apply -f emptydir-pod.yaml

pod/test-vol1 created
```

Step 3: Verify the Pod and Volume

Check the status of the Pod:

```
kubectl get pods
```

Access the Pod's shell:

```
kubectl exec -it test-vol1 -- bash
```

Inside the Pod, create a file in the /data directory:

```
touch f1 f2 f3
```

You should see the text "Hello, Kubernetes!".

Exit the Pod shell:

```
exit
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl get pods

NAME              READY   STATUS                       RESTARTS   AGE
pod-with-secret   0/1     CreateContainerConfigError   0          53m
test-vol1         1/1     Running                      0          58s
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl exec -it test-vol1 -- bash

root@test-vol1:/# touch f1 f2 f3
root@test-vol1:/# cd /data
root@test-vol1:/data# echo "Hello, Kubernetes!" > test.txt
root@test-vol1:/data# cat test.txt
Hello, Kubernetes!
root@test-vol1:/data# exit
exit
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

Step 4: Create a Pod with a hostPath Volume

Create a file named **hostpath-pod.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-vol2
spec:
  containers:
    - image: coolgourav147/nginx-custom
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      hostPath:
```

```
    path: /test
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % cat hostpath-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-vol2
spec:
  containers:
    - image: coolgourav147/nginx-custom
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      hostPath:
        path: /test
```

In this manifest:

The hostPath volume mounts a directory from the host into the Pod at /mnt/data.

The busybox container writes the current date to a log file every 5 seconds.

Apply the manifest to create the Pod:

```
kubectl apply -f hostpath-pod.yaml
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl apply -f hostpath-pod.yaml

pod/test-vol2 created
```

Step 3: Verify the Pod and Volume

Check the status of the Pod:

```
kubectl get pods
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl get pods

NAME              READY   STATUS                       RESTARTS   AGE
pod-with-secret   0/1     CreateContainerConfigError   0          58m
test-vol1         1/1     Running                      0          6m3s
test-vol2         0/1     ContainerCreating            0          5s
```

Access the Pod's logs to see the data being written:

```
kubectl exec -it test-vol2 -- bash
```

```
root@test-vol2:/# cat /data/hello.txt
Hello, Kubernetes! This is coming from inside the Pod.
root@test-vol2:/# exit
exit
```

You should see the dates being appended to the log file.

Check the host directory to verify the volume:

If you have access to the node running the Pod, you can verify the data in /tmp/data/log.txt.

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl exec -it test-vol2 -- bash

root@test-vol2:/# df -h | grep /data
overlay         392M  2.7M  390M   1% /data
root@test-vol2:/# exit
exit
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```